

CRC-16 Transmitter and Receiver on FPGA

Reconfigurable Computing (CS G553) — Lab Assignment 2, Question 2

Agnibha Sarkar (2023B5A31182G)

Avnet ZedBoard, Xilinx Zynq-7000 xc7z020clg484-1, 100 MHz, Vivado 2025.2

1 Objective

Implement a CRC-16 transmitter and receiver on FPGA. The transmitter accepts a variable-length message of up to 32 bits, supplied one bit at a time through a VIO, appends the CRC and displays the codeword being sent. The receiver asserts **RECEIVED** or **CORRUPTED**. An OVERRIDE multiplexer allows the receiver to be fed either from the transmitter or from a separate user-supplied stream. FSMD style throughout; no IP cores other than the VIO.

2 Algorithm

A CRC treats a bit string as a polynomial over $\text{GF}(2)$, where addition and subtraction are both XOR, so long division needs no borrows and no magnitude comparison. For a message $M(x)$ of degree $L - 1$ and a generator $G(x)$ of degree 16,

$$R(x) = M(x) \cdot x^{16} \bmod G(x), \quad T(x) = M(x) \cdot x^{16} + R(x). \quad (1)$$

Shifting up by 16 makes room for the remainder; adding it back makes $T(x)$ an exact multiple of $G(x)$. This design uses CRC-16-CCITT, $G(x) = x^{16} + x^{12} + x^5 + 1 \equiv 0x1021$ with the x^{16} term implicit, INIT = 0xFFFF, MSB-first, no reflection, no final XOR.

2.1 Why it detects errors

The receiver does not recompute and compare. It divides the *entire* received word — message and CRC together — by the same $G(x)$ and checks for a zero remainder. Writing the received word as $T'(x) = T(x) + E(x)$, and since $G(x) \mid T(x)$,

$$T'(x) \bmod G(x) = E(x) \bmod G(x). \quad (2)$$

The syndrome depends only on the error pattern, never on the data, and an error escapes if and only if $E(x)$ is a multiple of $G(x)$. Every guarantee follows: all single-bit errors are caught, since $E = x^i$ has no degree-16 factor; all odd-weight errors are caught, since $(x + 1) \mid G(x)$ and every multiple of $(x + 1)$ has even weight; all bursts of length ≤ 16 are caught, since $E = x^j B(x)$ with $\deg B < 16$; a 17-bit burst escapes in one case in 2^{15} ; longer random patterns escape with probability 2^{-16} , giving 99.9985% detection.

The initial value cancels. Seeding with I adds $I \cdot x^{L+16}$ at both ends, and over $\text{GF}(2)$ these XOR to zero, so the residue of a clean codeword is zero for any seed provided both ends agree.

2.2 Hardware form

Long division one bit at a time is a 16-bit shift register with taps. With R_k the remainder after k bits and d the next bit, $R_{k+1} = (R_k \cdot x + d \cdot x^{16}) \bmod G(x)$. Multiplication by x is a left shift; the bit shifted out, $R_k[15]$, is the coefficient at x^{16} , and combined with d it forms the quotient bit of the division step, $fb = d \oplus R_k[15]$. If $fb = 1$ the generator is XORed back in; otherwise it is not.

```
wire fb = din ^ crc[15];
always @(posedge clk) begin
    if (rst)      crc <= INIT;
    else if (en)  crc <= {crc[14:0], 1'b0} ^ (fb ? G : 16'h0000);
end
```

Sixteen flip-flops and three XOR gates, one bit per clock. The same module serves the transmitter (fed the message, producing $R(x)$) and the receiver (fed message and CRC, producing the residue), so it is written once and instantiated twice — the two ends agree by construction rather than by review.

3 Architecture

Data flows: VIO $\rightarrow$ serializer $\rightarrow$ transmitter $\rightarrow$ memory $\rightarrow$ channel $\rightarrow$ MUX $\rightarrow$ receiver, with the custom buffer feeding the MUX's second input.

Module	Role
CRC16.v	Shared Galois LFSR primitive; 16 FFs, 1 bit per clock
SERIALIZER.v	48-bit shift register; one parallel VIO write becomes a 1-bit-per-clock stream
CRC_TX.v	FSM LOAD→APPEND→DONE; absorbs the message, appends $R(x)$
MSG_MEM.v	Codeword store; parallel write, non-destructive serial read; exports <code>rd_index</code>
CustomBuff.v	Second source — a codeword typed by the user, no CRC appended
CRC_CORE.v	Structural glue, edge detection, error-injecting channel, OVERRIDE mux
CRC_RX.v	FSM RECV→CHECK→DONE; verdict from the residue
CRC_TOP.v	Pin mapping, two-flop synchronisers, VIO instance, LEDs

Serial entry. The handout permits supplying inputs sequentially either with a MUX addressed by a counter or with a shift register loaded from the VIO; this design takes the second option. The user writes a complete word and its length once, and the serializer left-aligns it so the MSB sits at the shift tap and emits it at exactly one bit per clock. The transmitter’s interface is strictly serial — the parallel-to-serial conversion is hardware, not a shortcut.

Variable length. `par_len[5:0]` sets 1–32 bits at runtime, `cw_len` is `msg_len+16`, and every read pointer is initialised from the stored length. No rebuild is needed to change the message length.

Channel. The memory exports the index of the bit currently being read, so the channel is one XOR gate, `err_bit = err_en & mem_valid & (mem_index == err_index)` — a literal hardware realisation of $E(x) = x^i$. The stored codeword is never modified; the bit is flipped in flight, so the VIO still displays the transmitted word while the receiver sees the corrupted one.

OVERRIDE mux. Selects the transmitter’s codeword from the memory, or a codeword typed by hand into `CustomBuff`. The second path demonstrates that the receiver is a true polynomial divider rather than a comparator — nothing in the design computed a CRC for a hand-typed word — and it is the only way to inject multi-bit and burst patterns that the single-bit channel cannot produce.

Control. Five asynchronous board inputs — BTNL `par_load` (N15), BTNR `start_tx` (R18), BTND `rst` (R16), SW0 `override` (F22), SW1 `load_sel` (G22) — pass through two-flop synchronisers and are edge-detected into single-cycle pulses, which the VIO requires because its JTAG-written registers hold their value for the order of 10^8 cycles. LD0–LD5 show RECEIVED, CORRUPTED, `tx_ready`, `rx_done`, `override` and `load_sel`. All five inputs and six outputs are declared `set_false_path`.

4 Resources consumed

Post-implementation, Vivado 2025.2, fully routed: 2644 of 2644 routable nets routed, zero routing errors.

Resource	Used	Avail.	%	Instance	LUT	FF	Slice	F7/F8
Slice LUTs	1197	53,200	2.25	<code>crc_top</code> (total)	1197	2006	546	7/3
as Logic	1173	53,200	2.20	<code>u_core</code> (design)	271	320	121	7/3
as Dist. RAM	24	17,400	0.14	<code>u_vio</code> (debug)	477	933	240	0/0
Slice Registers	2006	106,400	1.89	<code>dbg_hub</code> (debug)	449	741	218	0/0
Occupied Slices	546	13,300	4.11	top-level synchronisers	0	12	—	0/0
Block RAM	0	140	0.00	User logic	271	332		
DSP48E1	0	220	0.00	Debug infrastructure	926	1674		
Bonded IOB	12	200	6.00					
BUFGCTRL / BSCANE2	2 / 1	32 / 4	—					

User logic is 271 LUTs and 332 flip-flops — `u_core` plus twelve top-level synchroniser registers, or 0.51 % of the LUTs and 0.31 % of the registers on the XC7Z020. The VIO core and JTAG debug hub that Vivado inserts automatically account for 926 LUTs and 1674 flip-flops: $3.4\times$ the logic and $5.0\times$ the registers of the circuit they observe. The headline 2.25 % utilisation figure is therefore mostly debug infrastructure, not CRC.

Counted by hand from the RTL the register total is 343 (LFSRs 32, serializer 57, `crc_tx` 111, `msg_mem` 62, `custom_buff` 61, `crc_rx` 5, glue 5, synchronisers 10). Implementation reports 332, so synthesis trimmed eleven registers that no path can reach given `MAX_MSG = 32`.

Neither BRAM nor DSP is inferred, which is correct: the buffers are single 48-bit registers with 48:1 read multiplexers, not memories, and at this size fabric flip-flops are cheaper than a block RAM. All 7 F7 and all 3 F8 multiplexers in the entire design sit inside `u_core` — these are the dedicated wide-mux primitives

Vivado uses to build those read ports out of LUT6s. The CRC engine itself is negligible at 16 flip-flops and 3 XOR gates; the area is dominated by the 48-bit buffers, their read multiplexers, and above all the barrel shifter `sr <= par_data << (WIDTH - par_len)` in the serializer.

No DRC or methodology violation originates in user logic. The five routed-DRC warnings and four methodology warnings all lie inside `dbg_hub`, apart from ZPS7-1 (“PS7 block required”), which is expected for a programmable-logic-only design on a Zynq part.

5 Latency and throughput

Cycle counts were measured by simulation (`TB_LATENCY.v`) rather than estimated. L is the message length in bits and $N = L + 16$ the codeword length, at 100 MHz.

Phase	Cycles	$L = 32$	L	N	Load	Tx+chk	Total
Load: serialise, CRC, append, store	$L + 5$	37 cyc / 370 ns	4	20	9	22	31 cyc
Transmit: stream, divide, decide	$N + 2$	50 cyc / 500 ns	8	24	13	26	39 cyc
End-to-end	$2L + 23$	87 cyc / 870 ns	16	32	21	34	55 cyc
			32	48	37	50	87 cyc

The fixed overheads are small and constant — five cycles on the load side (serializer pipeline, the APPEND state, the memory write) and two on the receive side (CHECK, then registering the verdict). Latency is linear in message length because the datapath is bit-serial by design.

Throughput measure	Value
CRC engine and channel, peak	1 bit/clock = 100 Mbit/s at 100 MHz
Payload during the transmit phase, $L = 32$	32 bits / 500 ns = 64 Mbit/s
Payload end-to-end, $L = 32$	32 bits / 870 ns = 36.8 Mbit/s
Code rate $L/(L + 16)$	32/48 = 0.67
Peak rate at the achieved $F_{\max}$	158 Mbit/s

The LFSR never stalls, so raw throughput equals the clock rate. The end-to-end figure is lower for two structural reasons: sixteen of every $L + 16$ transmitted bits are check bits rather than payload, and the load and transmit phases are serialised, because the memory must hold the complete codeword before streaming begins so that the OVERRIDE mux and the error injector can address it by index. Overhead is a fixed 16 bits regardless of L , so the code rate rises from 67 % at $L = 32$ to 96 % at $L = 512$; the 32-bit cap is the assignment’s, not the architecture’s.

6 Timing closure and power

Timing metric	Value	Critical path
Constraint (<code>sys_clk</code>)	10.000 ns / 100 MHz	Source <code>u_vio → par_len[3]</code>
WNS (setup)	+3.684 ns	Destination <code>u_ser/sr_reg[39]/D</code>
TNS / failing endpoints	0.000 ns / 0 of 3636	Data delay 6.296 ns
WHS (hold)	+0.077 ns	logic 1.672 ns (26.6 %)
THS / failing endpoints	0.000 ns / 0 of 3620	routing 4.624 ns (73.4 %)
WPWS / failing endpoints	+3.750 ns / 0 of 2056	Logic levels 5 (LUT6×2, LUT5×2, LUT4)
F_{max}	158.3 MHz	Total power 0.119 W (0.013 W dynamic)

All user-specified timing constraints are met, with 58 % of the budget unused at 100 MHz. The worst path runs `par_len[3] → sr[2]_i_4 → sr[47]_i_17 → sr[41]_i_3 → sr[39]_i_2 → sr[39]_i_1 → sr_reg[39]`, and every intermediate LUT belongs to the serializer’s variable left-shift. The barrel shifter is therefore simultaneously the largest and the slowest block in the design, while the CRC engine is neither: its own path, `fb = din ⊕ crc[15]` followed by one XOR into the generator, is two levels of logic. Routing accounts for 73 % of the delay, characteristic of a small design placed thinly across two clock regions.

`check_timing` lists five input and six output ports with no I/O delay but with a false-path constraint — exactly the five board inputs and six LEDs declared in the XDC — and nothing else unconstrained. Dynamic power

by hierarchy is `u_vio` 0.005 W, `dbg_hub` 0.003 W and `u_core` 0.002 W; as with area, the debug infrastructure costs several times more than the circuit under test. Vivado rates the confidence of the power estimate *Low*, as no switching-activity file was supplied.